NSR + DriftSlice: unchanged-core optimization

Result-based arithmetic, native AIHWKit checks and limits · 23 October 2026

Conclusion: existing-parameter tuning has evidence for only a small improvement so far. It does not support a prediction above 90% at five years. No NSR loss, DriftSlice estimator, model checkpoint or original result was changed.

Freeze the completed NSR best-validation checkpoint, lambda=0.1, BatchNorm, input ranges, noise settings and programming policy. Keep target-class assignment, the ratio-of-means gain, branchwise correction and gain bounds [0.5,3]. The earlier least-squares proposal is excluded because it would change the estimator.

Full three-repetition / 30-realization test data: NSR + DriftSlice B=1 starts at 89.8828% and reaches 85.3043% at five years. The gap to 90% is 4.6957 percentage points. Restoring the initial accuracy alone would still be just below 90%; this is not a strict upper bound on every possible calibration change.

Actual reference-budget pilot: validation only, fold 0, three PCM realizations. B=1 and K=4 throughout.

Reference cells R	20s accuracy	5y accuracy	Retention loss
64	89.9067%	86.2467%	3.6600 pp
256	90.0467%	86.4267%	3.6200 pp

R=256 adds 0.18 percentage points at five years, but 0.14 points are already present at 20 seconds. Retention loss decreases by only 0.04 points. Five-year cross-entropy slightly worsens (0.675114 to 0.677818). This is a weak exploratory signal, not a robust demonstration.

Best-supported candidate to confirm: B=1, R=256, K=4, gain_max=3, with NSR unchanged. It uses 4x the reference cells and reference-read count of R=64. Confirm folds 1 and 2 before deciding whether the gain is worth that cost.

The mathematical error budget

Same algorithm; analysis of what the existing parameters can change

$$G_i(t) = G_{i,prog}\exp[-\nu_i \ell_t], \quad \ell_t = \log(t/20\,s)$$

$$\log[\widetilde{G}_i(t)/G_{i,prog}] = \log a_{b(i)}(t) - \nu_i \ell_t$$

A single class gain cannot remove every device-specific drift exponent. More reference cells improve knowledge of the class gain; they do not measure or correct each main-array device exponent.

$$c_{device} \approx \sqrt{\exp[(\sigma_\nu \ell_t)^2] - 1}$$

At five years, log(t/20s) = 15.881031. With the high-conductance nominal drift SD 0.008, the lognormal approximation gives relative device variation of 12.76%. This is not a network-accuracy bound.

Your 90 recorded five-year B=1 reference gains have mean 2.171653 and sample SD 0.044484: relative SD 2.05%.

$$V \approx c_{device}^2 + c_{gain}^2 + V_{other}, \quad c_{gain}^2 \propto 1/R$$

In this illustrative independent small-error model, increasing R from 64 to 256 divides gain variance by four. Omitting other errors, the combined variance decreases by approximately 1.89%. That does not mean an accuracy increase of 1.89 percentage points.

$$\mathrm{Var}(\log a_b) \approx A_b/R_b + D_b/(R_b K), \quad R_b = R/B$$

Increasing K averages read noise but not the persistent reference drift exponents. Increasing B narrows conductance classes but leaves fewer reference cells per class at fixed R. The constants also change with class, so increasing B is not guaranteed to help.

The initial and aged reference reads use the same programmed devices and are correlated. The ratio-variance calculation must include their covariance. The native simulation preserves that shared state; the Markdown report gives the full first-order expression.

Native PCM screen: small gains, not an accuracy forecast

128 independent states x 513 synthetic weights x 2 distributions · AIHWKit 1.1.0 CPU

The unchanged P4 conductance converter, programmed-conductance bounds, drift/read noise, reference bank and branchwise correction were used. Synthetic targets were uniform and concentrated near zero. Main-array states and read seeds were paired across candidate settings. User-trained checkpoint weights were not available locally.

B	R	K	R x K reads	MSE change
1	64	4	256	+0.00 to +0.00%
1	64	16	1024	-0.27 to -0.27%
1	256	4	1024	-1.53 to -1.43%
2	256	4	1024	-1.31 to -0.95%
4	256	4	1024	-0.37 to +0.45%
1	4096	4	16384	-1.85 to -1.69%

Negative MSE changes are better. These are relative changes in reconstructed logical-weight mean-squared error, not accuracy percentage points. R=256 modestly helps both synthetic distributions; increasing K alone helps less; B=4 is not consistently better.

Even 4096 reference cells have limited benefit in this model screen despite 64x the baseline reference area and read count. This is consistent with a substantial main-array error component that reference-bank precision cannot remove.

Accuracy also depends on the trained network, output margins, BatchNorm, quantization and saturation. Aggregate accuracy/loss/gain CSVs cannot reconstruct the logits for an untested configuration. A new validation inference run is required to measure its accuracy.

MLP parity and practical next step

Shared core equation checks passed; complete experimental protocols are not identical

Fresh structural checks matched the original uploaded MLP NSR formula, conductance edges, class assignment, reference-read averaging, ratio-of-means gains and branchwise correction. On the same native AIHWKit MLP weights, NSR scalar values and physical-weight gradients were bit-identical.

Direct native checks verified the gain arithmetic, reference budget and branchwise correction at several B/R/K settings. Programmed conductances and drift exponents were unchanged by reads. The synthetic PCM screen reproduced exactly after making its script portable.

Important existing difference: the supplied full MLP definition defaults to nonnegative-only programming bounds, while P4 also caps programmed conductance at gmax. Its saved MLP manifest was not supplied, so the actual MLP run policy cannot be confirmed. This work does not modify either policy or conceal that difference.

Use the provided confirmation cell in the initialized P4 runtime. It evaluates only validation data for R=64 versus R=256 on folds 1 and 2, with three device realizations and two timestamps. It reads the original completed HWA and NSR best-validation checkpoints and writes to a separate pilot directory. No training or official-test evaluation is run.

If the small R=256 gain does not persist across the remaining training repetitions, retain R=64. If it does, record the 4x reference cost and evaluate the selected configuration under a new retention identity. Apply equal resources to HWA when assessing the incremental NSR contribution.

Nothing in these checks supports a guaranteed five-year accuracy above 90% under the strict unchanged-core constraint. The conclusion is a measured resource-tuning candidate, not a new proven algorithm or a hidden change to the MLP method.